CAS Memory Substrate PDR and Detailed Design

CAS Memory Substrate: PDR and Detailed Design

Date: 2026-04-22 Status: Future-work architecture proposal, not current production capability Primary question: Can CAS address AI and agent memory in a meaningful, effective way?

Executive Verdict

The tested claim is directionally true but needs narrower wording.

Current project memory tools, including git artifacts, CLAUDE.md, changelog continuity, ADR-style documents, and session transcripts, are adequate for project continuity. They do not constitute a fundamental AI memory architecture.

CAS does provide a plausible and unusually rich substrate for cognitive memory: the 40 Cognitive States, Phenomenological Registers, Drives, RPM prerequisite chains, lacunarity, named emergent registers, and production semantics can act as a cognitive indexing ontology over ordinary memory infrastructure.

The strongest corrected claim is:

> CAS does not replace existing memory plumbing such as vector stores, archival memory, graph databases, or agent-managed memory blocks. CAS can provide the cognitive schema that rides on top of those systems, turning text retrieval into state-aware, prerequisite-aware, gap-aware associative recall.

Confidence assessment:

- Immediate practical memory verdict: high confidence.
- "Current arsenal does not solve AI memory fundamentally": high confidence.
- "CAS could ground a better memory system": medium-high confidence.
- "This exists nowhere else": too strong as written; medium-low confidence.
- "This is substantial future work": high confidence.

Evidence Basis

Local repository evidence:

- `CLAUDE.md` defines the 40 Cognitive States, 7 Drives, and Register as a specification layer over existing primitives.
- `cas-native/model/expression_proposer.py` implements ADR-07 `ExpressionProposerHead`, a learned head that proposes structured CAS expressions from hidden states.
- `cas-native/training/data_format.md` defines v2 label streams for register labels, Omega labels, and production targets.
- `cas-native/config/omega_registry.yaml` provides the admitted Omega registry.
- `cas-native/model/lacunarity_gate.py`, `cas-native/model/lacunarity_head.py`, `api/pipeline/lacunarity.py`, and `cas-engine/src/scoring/lacunarity.rs` implement lacunarity concepts.
- `cas-engine/src/rpm/` and `cas-native/model/rpm_gate.py` implement RPM concepts.

External memory-system context:

- Letta/MemGPT-style systems include core memory, archival memory, and agent-managed memory hierarchy.
- LangGraph documents short-term and long-term memory, including semantic, episodic, and procedural memory categories.
- Generative Agents used memory, reflection, and planning as a behavioral substrate.

Finding: current AI memory systems have meaningful structure, so the statement "none of this exists elsewhere" is too broad. CAS is distinctive because it offers a specific cognitive-coordinate ontology rather than because other systems have no memory architecture.

Simulated Codex/Gemini Design Debate

Note: This is a simulated internal design debate using the Codex ML-engineering position and the Gemini architecture-generalist position supplied in the session. It is not an external Gemini model invocation.

Round 1: What Problem Are We Solving?

Codex position:

The immediate problem is not storage. Storage, retrieval, and persistence are already solved well enough by existing systems. The missing layer is a reliable way to decide what a memory means, when it should be retrieved, and what gaps remain. CAS can help with encoding, retrieval reranking, and evaluation, but it does not solve write policy, disk storage, conflict resolution, or forgetting.

Gemini-style position:

The distinction is mechanics versus meaning. Standard memory systems are filing cabinets: context windows, archival tiers, vector similarity, and explicit memory blocks. CAS is closer to a cognitive classification system: it can identify whether a memory is analytical, affective, embodied, prerequisite, productive, absent, or emergent. The proper design is not CAS instead of memory plumbing, but CAS as a cognitive schema over memory plumbing.

Synthesis:

CAS Memory Substrate should be built as an overlay architecture. It must not try to replace vector stores or graph databases.

Round 2: What Should The First Real Artifact Be?

Codex position:

Start with a thin CAS Memory Ledger. Each memory record needs content, provenance, timestamp, CAS state metadata, salience, confidence, and status. Keep the first version deliberately small. Avoid building a full graph memory system before CAS-Native v2 is validated.

Gemini-style position:

A flat ledger is useful only as an intermediate interface. CAS memory is intrinsically relational: RPM prerequisites and production semantics are edges, not fields. The target data model should be graph-first with vector embeddings attached to nodes. Memories should be nodes; prerequisite, supports, refutes, produces, and fills-gap relationships should be edges.

Synthesis:

Use a staged path:

1. V0: file-backed or table-backed CAS Memory Ledger for experimentation.
2. V1: hybrid graph plus vector store once the schema stabilizes.
3. V2: active memory agent that writes, updates, retrieves, and audits memory.

Round 3: What Is The Highest-Leverage CAS Feature?

Codex position:

The highest-leverage immediate feature is automated CAS tagging. ADR-07 `ExpressionProposerHead` can become the bridge from hidden states to structured memory metadata once trained. This means memory annotation can reuse a model component already aligned with CAS-Native v2.

Gemini-style position:

The highest-leverage differentiated feature is lacunarity. Ordinary vector databases retrieve what exists. CAS can detect what is structurally missing: for example, a memory set may contain many analytical memories about a topic but no embodied/practical examples.

Synthesis:

The first nontrivial CAS memory prototype should combine:

- ADR-07 proposer as an auto-tagger.
- CAS-state metadata attached to memory records.
- Lacunarity reports over memory coverage.

This avoids building every memory lifecycle component while proving the unique value of CAS memory.

Round 4: What Should We Not Claim?

Codex position:

Do not claim CAS memory is ready. Do not claim existing memory systems have no structure. Do not imply CAS solves persistence, forgetting, update policy, or conflict handling.

Gemini-style position:

Do claim that CAS provides a unique cognitive ontology for memory organization. Frame it as an unlock from CAS v7.7+ and CAS-Native v2, not as a current deployment feature.

Synthesis:

Document CAS Memory Substrate as high-value future work unlocked by the current architecture. Do not divert CAS-Native v2 training work to build it now.

Product Design Requirements

Goal

Design a CAS-structured memory substrate that can layer on top of existing agent memory infrastructure and improve memory encoding, retrieval, gap detection, and evaluation.

Non-Goals

- Do not replace vector databases, archival memory, or normal persistence.
- Do not build a production memory system before CAS-Native v2 is validated.
- Do not introduce new CAS primitives.
- Do not treat memory annotations as ground truth without confidence scores.

- Do not make memory writes irreversible.

Users

- Agent runtime needing durable, queryable memory.
- Researcher evaluating CAS-Native v2 memory behavior.
- Developer maintaining project continuity across sessions.
- Future CAS Synthetic Data Generator producing memory-annotated examples.

Functional Requirements

1. Memory records must store source content, provenance, timestamp, and version.
2. Memory records must support CAS annotations:
   - Noetic indices or labels.
   - Domain indices or labels.
   - Register labels.
   - Drive or Foundation salience.
   - RPM prerequisite links.
   - Production links.
   - Lacunarity coverage fields.
   - Optional Omega tags.
3. Retrieval must support ordinary semantic search.
4. Retrieval must support CAS-aware reranking.
5. Retrieval must support prerequisite expansion.
6. The system must report confidence and provenance for retrieved memories.
7. The system must detect memory lacunarity across CAS coverage.
8. The system must allow stale or contradicted memory to be revised.
9. The system must keep CAS annotations separate from raw memory content.
10. The system must be compatible with existing CAS-Native v2 outputs.

Nonfunctional Requirements

- Explainability: every retrieved memory should state why it was selected.
- Auditability: writes and updates should be traceable.
- Incrementality: V0 must work without a graph database.
- Portability: memory export should support JSONL.
- Safety: sensitive memory should support redaction and deletion.
- Performance: V0 retrieval should remain practical for thousands of memories.
- Compatibility: no change to v7.4 byte-equivalent training paths.

Acceptance Criteria

V0 acceptance:

- A memory record schema exists and validates.
- CAS annotations can be attached manually or by an auto-tagger stub.
- Retrieval can combine text similarity and CAS metadata filters.
- A lacunarity report can identify undercovered CAS coordinates.
- A small project-memory corpus can be exported and reloaded.

V1 acceptance:

- ADR-07 proposer can generate candidate CAS tags.
- Graph edges represent RPM prerequisite and production relationships.
- Retrieval can pull prerequisite context automatically.
- Evaluation shows CAS-aware retrieval improves task outcomes over vector-only retrieval on a held-out memory benchmark.

Detailed Design

Architecture Overview

CAS Memory Substrate has five layers:

1. Memory capture layer.
2. CAS annotation layer.
3. Storage layer.
4. Retrieval and reasoning layer.
5. Evaluation and lacunarity layer.

Data flow:

```
event or artifact
  -> memory candidate
  -> CAS annotation
  -> memory ledger / graph store / vector store
  -> query
  -> semantic retrieval
  -> CAS rerank
  -> prerequisite expansion
  -> lacunarity audit
  -> context packet
```

V0 Memory Record

V0 should use a JSONL-compatible schema so it can live in git or a simple database table.

```
{
  "id": "mem_20260422_0001",
  "content": "Panel converged on Option E for Register FiLM alignment.",
  "summary": "Option E accepted as shared register latent with dual heads.",
  "source": {
```

```
    "kind": "session",
    "path": "session_transcript",
    "span": null
  },
  "created_at": "2026-04-22T22:00:00-07:00",
  "updated_at": "2026-04-22T22:00:00-07:00",
  "cas": {
    "noetics": ["Integration", "Structure"],
    "domains": ["Cognitive", "Operational"],
    "registers": ["A"],
    "drives": ["Wisdom", "Structure"],
    "rpm_prerequisites": [
      "Round 1 option votes",
      "Round 2 A-vs-C convergence"
    ],
    "omega_tags": [],
    "production_roles": [],
    "lacunarity_notes": ["Needs implementation ADR"]
  },
  "salience": 0.92,
  "confidence": 0.95,
  "status": "active",
  "supersedes": [],
  "superseded_by": [],
  "embedding_ref": null
}
```


V1 Graph Model

Node types:

- Memory
- Topic
- CASState
- Register
- Drive
- RPMNode
- ProductionRecipe
- OmegaCategory
- Artifact
- Session

Core edges:

- `MENTIONS`
- `HAS_CAS_STATE`
- `HAS_REGISTER`

- HAS_DRIVE
- REQUIRES
- SUPPORTS
- REFUTES
- SUPERCEDES
- PRODUCES
- FILLS_GAP
- DERIVED_FROM
- ADMITTED_AS

Retrieval should start with vector similarity, then traverse graph edges for CAS-relevant expansion.

CAS Annotation Layer

V0 annotation modes:

1. Manual annotations from developer/user.
2. Rule-based annotation from existing CAS pipeline scores.
3. Stubbed proposer interface for future ADR-07 integration.

V1 annotation modes:

1. ADR-07 `ExpressionProposerHead` proposes Noetic, Domain, Register, operand count, and named-emergent structure from hidden states.
2. CAS pipeline validates and normalizes the annotation.
3. Human or governance workflow can approve high-salience memory categories.

Important constraint: ADR-07 proposals are candidate metadata, not guaranteed truth. Store confidence and source.

Retrieval Design

Retrieval pipeline:

1. Query embedding search returns top N candidate memories.
2. CAS query parser extracts target Noetics, Domains, Registers, Drives, and RPM dependencies from the query.
3. CAS reranker scores each memory:

```
score =
  w_semantic * semantic_similarity
  + w_state * cas_state_match
  + w_register * register_match
  + w_drive * drive_salience
  + w_rpm * prerequisite_relevance
  + w_recency * recency_score
  + w_confidence * confidence
  - w_stale * stale_penalty
```

4. Graph expansion pulls required prerequisites and production-related memories.
5. Lacunarity audit reports what is missing from the retrieved context.
6. Context packet returns memories plus reasons.

Lacunarity Design

Memory lacunarity should measure undercoverage across:

- 40 Noetic x Domain coordinates.
- 4 Registers.
- 7 Drives or 28 Foundations.
- Topic-specific prerequisite chains.
- Production roles.

Example output:

```
{
  "topic": "CAS memory substrate",
  "coverage": {
    "analytical": 0.84,
    "embodied": 0.12,
    "affective": 0.18,
    "transcendent": 0.31
  },
  "largest_gap": "Embodied/Operational implementation examples",
  "recommended_query": "Ask for practical deployment cases and failure modes."
}
```

This is the strongest differentiated feature: the agent can ask for memories it does not yet have, rather than only retrieving memories it already has.

Memory Lifecycle Mapping

CAS contribution by lifecycle stage:

1. Write: partial contribution. CAS can score salience but should not decide all writes alone.
2. Encode: strong contribution. CAS tags are the main value.
3. Store: weak contribution. Use existing storage systems.
4. Retrieve: strong contribution. CAS reranking and graph expansion matter.
5. Update: partial contribution. CAS can detect conflict type, but policy is separate.
6. Forget/archive: partial contribution. Lacunarity and salience help, but retention policy is separate.
7. Evaluate: strong contribution. CAS coverage and task outcomes can be measured.

Evaluation Plan

Benchmarks:

1. Vector-only retrieval baseline.
2. Vector plus CAS reranking.
3. Vector plus graph prerequisite expansion.
4. Full CAS memory with lacunarity audit.

Metrics:

- Task success rate.
- Retrieval precision and recall.
- Prerequisite completeness.
- User correction rate.
- Staleness/conflict rate.
- Lacunarity reduction over time.
- Explanation quality.
- Latency.

Test scenarios:

- Project continuity across sessions.
- Multi-step architecture decision recall.
- Contradictory memory update.
- Missing prerequisite detection.
- Practical example gap detection.
- Omega category admission history.

Implementation Roadmap

Wave M0: Documentation and schema only.

- Commit this PDR and design.
- Define memory record schema.
- Define benchmark tasks.

Wave M1: Thin ledger prototype.

- JSONL or SQLite-backed memory ledger.
- Manual CAS annotations.
- Text search plus metadata filters.
- Basic lacunarity report.

Wave M2: CAS auto-tagging prototype.

- Integrate trained ADR-07 proposer when available.
- Store proposer confidence.
- Compare manual versus auto tags.

Wave M3: Hybrid graph/vector memory.

- Add graph database or graph-like relational layer.
- Represent RPM dependencies and production relationships as edges.
- Add prerequisite expansion retrieval.

Wave M4: Active memory agent.

- Write/update/forget policies.
- Conflict resolution workflow.
- Governance for Omega memory categories.
- Evaluation against held-out memory benchmark.

Risks and Mitigations

Risk: CAS tags are wrong or overconfident. Mitigation: store confidence, provenance, and tag source; allow revision.

Risk: CAS memory distracts from CAS-Native v2 training. Mitigation: keep memory work in future wave until v2 core is validated.

Risk: graph model becomes too complex. Mitigation: start with V0 ledger, graduate to graph only after schema stabilizes.

Risk: memory becomes unverifiable narrative. Mitigation: require source pointers and audit trails.

Risk: lacunarity reports hallucinated gaps. Mitigation: compute gaps from explicit coverage statistics, not generated text.

Risk: privacy and sensitive memory retention. Mitigation: include redaction, deletion, and status fields from V0.

Final Recommendation

Accept the revised verdict:

CAS does not currently solve AI memory. Current project tools should be used for immediate continuity. CAS Memory Substrate is a high-value future architecture that can add cognitive-state annotation, prerequisite-aware retrieval, production-aware linking, and lacunarity-based gap detection on top of existing memory infrastructure.

Do not implement the full system during the current CAS-Native v2 push. The right next step is to preserve this design, then build a thin ledger prototype after CAS-Native v2 and the Synthetic Data Generator have enough stability to produce reliable CAS annotations.